

SOYUT SINIFLAR HAKKINDA BİLMENİZ GEREKENLER

1. Soyut metod içeren sınıf, soyut olmak zorunda.
2. Soyut metod sadece soyut sınıfta/arayüzde bulunabilir.
3. Bir soyut sınıftan soyut metod, yanrı sınıflar gerçekleştirilmeli (overriding) aksi takdirde yanrı sınıf da soyut tanımlanmaz.
4. Soyut metodlar static tanımlanamaz.
5. Bir soyut sınıftan nesne yaratılamaz.
6. Soyut sınıflar soyut metod barındırmak zorunda değildir.
7. Bir yanrı sınıfta ata sınıf olan soyut sınıftan miras alınan (concrete) somut metod abstract bir metod ile eşlebilir. (çöz nedir)
8. Bir soyut sınıfın atası concrete (somut) sınıf olabilir.
9. Soyut sınıflardan new ile nesne yaratamayız ama yine de onları wrapper olarak kullanabiliriz.
10. Soyut sınıflardan yeni nesneler yaratmadığımız halde konstrüktör kullanılabilir.

```
ArrayList <Number> list = new ArrayList<>();  
if (list.size() == null)
```

! Ya null olan bir liste olursa, o zaman kod
Soldan sağa ulaşacağı için runtime hatası verecektir.
Bazılatılmayan listenin boyutu bilinemez!

Syf. 502

* Ata sınıfın final olması o sınıftan miras alınmayı engeller.

* Sınıfın adı italik ise abstract class.

→ protected

getter → accessor

setter → mutator

* Soyut metodun gövdesi olmaz.

(;) noktalı virgül ile bitmeli.

* abstract metodlarda bir şey döndürmesi (int, float) veya döndürmemesi (void) çok önemli değil.

```
list.add(45);
```

```
Integer x=45;
```

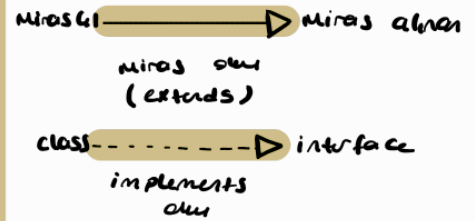
```
Integer x = new Integer(45);
```

↳ Yukarıdaki tek satırı yama ca
kısmı kendi kendine yaptı (Auto-boxing)

Interface (Arayüz): Deneyim
Taslağı

* Interface'ler "implement" edilir.

<< Interface >> → UML diyagramında



* Soyut sınıfların içinde somut metod olabilir.

* Interface'teki bütün metodlar public'tir. Bunları etkin de public abstract kullanmak gerekir.

* Java'da tek miras vardır. Birden fazla sınıftan miras alınmaz.

* Arayüzün konstrüktörü olmaz.

ARAYÜZLER HAKKINDA BİLMENİZ GEREKENLER

1. Arayüzler sadece sabitler ve soyut metodları barındırır. Concrete (somut) metod barındırmazlar.
2. Arayüzler de tipki soyut sınıflar gibi wrapper olarak kullanılabilir.
3. Arayüzlerden de tipki soyut sınıflardan olduğu gibi nesneler üretilmez.
4. Soyut sınıflar miras alır (extends), arayüzler gerçekleştirir (implements)
5. Bir sınıf sadece bir ata sınıftan (soyut/somut) miras alabilir, ama çok çok arayüzü implement edebilir.
6. Bir arayüzden miras alan tüm soyut metodlar ezilmelidir, aksi takdirde miraslı sınıfın abstract tanımlanması gerekir.
7. Arayüzlerde tanımlanan tüm değişkenler otomatikman "public static final" olur. Tüm soyut metodlar ise otomatikman "public abstract" olur.
8. Arayüzlerin konstrüktörü olmaz.
9. Arayüzler başka arayüzleri miras alabilir ama normal sınıflar miras almaz.

Comparable Interface

Cloneable Interface

* İki boş interface'le MARKER INTERFACE olarak gelir.

